SECURITY REVIEW — ROUND 2, REVISED

Smart Contract Security Review

mezo-paas — billing & collateral contracts

Scope	**Solidity contracts only — 682 lines, 2 files**
Primary target	`contracts/BillingContractV2.sol (672 lines)`
Review type	**Re-review of fixes, plus fresh vulnerability hunt**
Branch / commit	`master @ acb9bd9`
Live deployment	`0x1fdf4276…3aecbac99 (Mezo testnet)`
Verification	**55/55 project tests run; 16 custom PoCs executed**
Reviewed	**2026-09-28 — revised 2026-10-02**
Round 1 findings	**ALL 5 FIXED**
Round 2 findings	**ALL 3 FIXED**
Open findings	**None**

Round 1 fixes: confirmed genuine

All five findings from the 2026-09-15 review are fixed, verified by running the project's new 606-line test suite (17/17 pass) and by ten independent proof-of-concept tests written for this round. The V1 contract was deleted outright, the owner and treasury are now **distinct addresses** on-chain, and `emergencyWithdraw` can no longer touch user collateral.

Revision 2026-10-02 — all three round 2 findings are fixed

The High finding reported here (the early-withdrawal penalty leaving the contract on a payout that never happened) is resolved, along with both Mediums. Verified by running the project's expanded suite (**55/55 pass**) and by six independent proof-of-concept tests that re-ran the original exploits against the fixed contract.

The contract was redeployed to `0x1fdf4276…3aecbac99`. On-chain checks confirm the instant-drain function is gone, the treasury-move timelock reads 48 hours, and owner and treasury remain distinct addresses. **No findings remain open.**

Round 1 fixes — verified

Every finding from the first review was addressed in commit `6930e00` ("fix: fix issues from audit") and redeployed. Each was re-checked against the current source and confirmed by execution.

#	STATUS	ROUND 1 FINDING	HOW IT WAS FIXED
1	FIXED	~~Owner can drain all user collateral~~	`emergencyWithdraw` limited to `balance − liabilities`; two-step treasury change with 24h timelock
2	FIXED	~~Unbacked withdrawal queue strands funds~~	Permissionless `claimQueuedWithdrawal()`; re-lock blocked while pending
3	FIXED	~~Reserve ratio uses wrong base~~	Now computed against `totalLockedCollateral + totalPendingWithdrawals`
4	FIXED	~~Fee-on-transfer accounting mismatch~~	Pattern removed; standard-ERC-20-only documented at contract level
5	FIXED	~~Unauthenticated `harvestYield`~~	`BillingContract.sol` (V1) deleted from the repository

> **On-chain trust separation — the key round 1 fix**
>
> Round 1's most serious issue was that `owner` and `treasury` resolved to the same address, so a single key could drain everything. On the new deployment `0x6f0730cb…186ce5a2` they are verified distinct: owner `0x155c4e84…9655c92c`, treasury `0x912718a6…c132dcfa`. The new deploy script also hard-rejects a deployment where the two match.

Round 2 findings — all resolved

#	SEVERITY	FINDING	LOCATION	CONF.
6	FIXED	~~Early-withdrawal penalty is taken even when the payout is queued and never delivered~~	V2:252–261	—
7	FIXED	~~`pause()` blocks user self-claims while leaving owner payout paths open~~	V2:294	—
8	FIXED	~~Reserve policy still permits stranding 80–90% of user collateral~~	V2:365	—

All three were reported on 2026-09-28 and fixed in commit `879debf` ("fix: audit 2 report"). Severities shown are those assigned when each was open. Details of each fix follow.

Round 2 findings in detail

6. Early-withdrawal penalty is taken even when the payout is queued and never delivered

`contracts/BillingContractV2.sol:252–261` — **POC EXECUTED**

FIXED broken_accounting / fund_loss · originally High, confidence 0.90

DESCRIPTION

`withdrawCollateral` executes in the wrong order. The 5% penalty is irreversibly transferred to the treasury at line 258, and only afterward, at line 266, does the function check whether the contract holds enough to pay the user. If it does not, the user is queued and receives nothing — but the penalty has already left the contract.

The queued claim amount is recorded *net of* the penalty (line 269 stores `amountToReturn`, already reduced by 5%). So the user is charged an early-exit fee for a withdrawal that did not occur, and the fee is unrecoverable: no function refunds a penalty, and the user cannot cancel the queued request to undo it.

This is distinct from round 1's Finding 2. That finding was about the queue being owner-gated and was correctly fixed. This is about value leaving the contract on a code path that fails to deliver anything, which the fix did not address because the penalty transfer was never moved below the solvency check.

EXPLOIT SCENARIO

Confirmed by executed PoC against the current contract, with a fully-funded control case proving the bug is specific to the thin-reserve path:

```
user1 locks 1000, user2 locks 1000        -> contract holds 2000
owner moves 1600 to treasury (permitted)  -> contract holds  400
user1 withdraws EARLY:
  penalty 50 -> treasury        (transfer succeeds)
  payout 950 -> contract has 400, cannot pay
  user1 queued for 950, receives 0

  user1 token delta   : 0
  treasury gained     : 50
  remaining liabilities: 1950   remaining balance: 350
  SHORTFALL           : 1600
```

The control case — same early withdrawal against a fully funded contract — pays out correctly and does not queue, isolating the defect to the ordering rather than to penalty logic in general. A user in this state has paid 50 tokens and holds a claim that the contract cannot honor until the treasury returns funds voluntarily.

> **HOW IT WAS FIXED — VERIFIED 2026-10-02**
>
> Implemented as recommended. The penalty is computed first and the solvency check now runs *before* any transfer, against a `freeBalance` that excludes funds already owed to queued withdrawals. On the queued path nothing leaves the contract: the penalty is stored in a new `penalty` field on `WithdrawalRequest` and settled by `_settleQueuedWithdrawal` at claim time, so fee and payout succeed or fail together.
>
> Re-running the original exploit against the fixed contract: treasury delta **0**, user delta **0**, with the 50-token penalty held on the request rather than paid. On claim the user receives 950 and the treasury receives 50, in the same transaction.

7. `pause()` blocks user self-claims while leaving owner payout paths open

`contracts/BillingContractV2.sol:294` — **POC EXECUTED**

FIXED access_control / availability · originally Medium, confidence 0.85

DESCRIPTION

`claimQueuedWithdrawal()` carries `whenNotPaused`, but `processQueuedWithdrawal()` (owner-only) does not. Pausing therefore removes the user's permissionless exit — the central remedy introduced for round 1's Finding 2 — while leaving the owner fully able to move funds. `emergencyWithdraw` is likewise unaffected by pause.

The result is that the round 1 fix holds only while the owner chooses not to pause. A permissionless claim that an administrator can unilaterally disable is not fully permissionless, and the asymmetry favors the owner at precisely the moment users most need the exit.

EXPLOIT SCENARIO

Confirmed by executed PoC. A whale's withdrawal is queued and the treasury returns enough to cover it, so the claim would succeed. The owner calls `pause()`. The user's `claimQueuedWithdrawal()` now reverts with `EnforcedPause` despite the funds sitting in the contract. The owner then calls `processQueuedWithdrawal()` successfully while still paused — demonstrating that pause constrains only the user. A compromised or uncooperative owner can hold users' matured claims indefinitely.

> **HOW IT WAS FIXED — VERIFIED 2026-10-02**
>
> `whenNotPaused` was removed from both `claimQueuedWithdrawal()` and `withdrawCollateral()`, with a comment recording the rule: pause gates entry points and outflows to treasury, never user exits.
>
> Confirmed by PoC: a user withdrew successfully while the contract was paused, and `lockCollateral` still reverted with `EnforcedPause`. The asymmetry now runs the correct way — pause restricts the owner and new obligations, not the people owed money.

8. Reserve policy still permits stranding 80–90% of user collateral

`contracts/BillingContractV2.sol:365` — **POC EXECUTED**

FIXED design / fund_availability · originally Medium, confidence 0.85

DESCRIPTION

Round 1's Finding 3 was fixed: the reserve is now correctly computed against real liabilities. The residual issue is the policy itself rather than the arithmetic. At the default 20% ratio the owner may move 80% of collateral to the treasury, and `updateReserveRatio` permits lowering the floor to 10%, allowing 90%. Once those funds are out, a withdrawing user is queued and `claimQueuedWithdrawal` reverts until the treasury voluntarily returns tokens.

The permissionless claim fixes owner *unavailability*, not owner *unwillingness or insolvency*. Because the treasury invests off-chain by design, there is no on-chain guarantee the funds still exist. This is a deliberate architectural choice, not an implementation error — it is recorded here because the residual trust assumption is substantial and is not stated anywhere users would see it.

EXPLOIT SCENARIO

Confirmed by executed PoC: with 1000 locked and 800 moved to treasury, a matured withdrawal is queued and the user's self-claim reverts with "Insufficient contract balance". A second PoC showed the contract balance reaching zero while a locked user still had a 1000-token claim outstanding. No attacker is required — ordinary operation of the documented yield strategy produces this state.

> **HOW IT WAS FIXED — VERIFIED 2026-10-02**
>
> Addressed more thoroughly than recommended. `moveCollateralToTreasury` was removed entirely and replaced by a propose/execute pair behind a **48-hour timelock** with a 7-day execution window, so users get notice to exit before a large move. The reserve floor is now a hard 20% (`MIN_RESERVE_RATIO_BPS`), and the ratio is re-checked at execution time against current liabilities.
>
> The disclosure gap was closed on-chain rather than only in documentation: a new `getLiquidityStatus()` reports contract balance, total liabilities, current backing in basis points, the guaranteed reserve floor, and any pending move with its execution time. A reserve-policy disclosure was also added to the contract header.
>
> Confirmed by PoC and by live on-chain reads: the old selector `0x1447cecc` now reverts, `TREASURY_MOVE_DELAY` returns 172,800 seconds (48 hours), and a user was able to exit during the notice window.

Tested and found sound

These were actively probed this round. Each held, and several were candidate findings that the tests disproved rather than confirmed — recorded so a later reviewer knows they were checked.

- `emergencyWithdraw` **cannot touch collateral.** With 1000 locked and no excess, the call reverts with "No excess funds to withdraw". The round 1 drain path is genuinely closed.
- **Treasury timelock cannot be shortened by re-proposing.** Re-proposing after 23 hours resets `treasuryChangeTimestamp` rather than inheriting the old one, so the full 24 hours always applies. This was a candidate High; the test disproved it.
- **Queued users cannot re-lock to fabricate a second vault.** The `"Pending withdrawal exists"` guard holds, closing the request-overwrite path from round 1.
- **No double-claim from the queue.** `isPending` and `amount` are both cleared before transfer, and the project's own test confirms a second claim reverts.
- `updateReserveRatio` **floor is enforced.** Setting 5% reverts; the 10% minimum holds, bounding how much can be moved out.
- **Reserve math tracks real liabilities.** After a full-payout withdrawal, contract balance still covers `totalLockedCollateral + totalPendingWithdrawals`.
- **Reentrancy.** All value-moving functions carry `nonReentrant`, and state is written before external calls throughout, including the new claim path.

> **Verification method**
>
> Dependencies were installed and the project's own suite was run: **17/17 pass**. That suite is organised by the round 1 finding numbers and is credible work.
>
> Because a fix suite written by the implementer tends to test the fixed path rather than adjacent ones, ten independent proof-of-concept tests were written for this round and run against the current contract. Those produced Findings 6, 7 and 8, and disproved three further candidates. All PoC files were removed afterward; `package-lock.json` was restored after `npm install` modified it, so the repository working tree is unchanged.

Status and remaining work

Every finding from both rounds is closed. Nothing in this report requires action.

- **Fixed beyond what was reported:** immediate payouts previously could consume funds earmarked for queued withdrawals. `withdrawCollateral` now computes a `freeBalance` net of `totalPendingWithdrawals` before paying anyone. This was not in the report; the team found and fixed it alongside Finding 6. A PoC confirmed a queued user can still claim after a later user withdraws.
- **Test coverage:** the suite grew from 17 to 55 tests, including a dedicated 622-line round 2 file and regression tests asserting the fixed behavior.

- **Standing recommendation, unchanged:** the reserve policy is now disclosed on-chain and gated by a timelock, but users still rely on the treasury returning funds it holds off-chain. That is an architectural property of the yield design, not a defect — worth restating in user-facing materials alongside the on-chain disclosure.
- **Before real capital:** get a professional third-party audit. Two internal rounds with every finding fixed and tested is a strong signal, not a certification.

> **Overall assessment**
>
> The response across both rounds was substantive rather than cosmetic: real architectural changes, a test suite grown from nothing to 55 tests, the dead V1 contract removed, the owner/treasury separation fixed on-chain, and — in round 2 — an instant-drain path replaced by a timelocked propose/execute flow with on-chain liquidity disclosure.
>
> All three round 2 findings are now fixed and independently verified, and the team resolved an adjacent accounting issue that was never reported to them. Fixes landed before the contract held any real collateral, which is the right order: the exploit paths were closed while the cost of closing them was still zero.
>
> This remains an internal review, not a professional third-party audit. Get one before real capital is at stake.